

Rendu individuel

Elyess Rjafellah

CI/CD & automatisation

Auteur	Elyess Rjafellah
Rôle	CI/CD & automatisation
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Chaîne GitHub Actions, état distant, pipeline auto-réparant

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

Ma mission a porté sur le cœur de la démarche DevOps : la chaîne d'intégration et de déploiement continu. L'objectif fixé collectivement était ambitieux — obtenir un déploiement « en un clic » où l'intégralité du cycle de vie s'exécute automatiquement, sans intervention manuelle, et se rétablit seul en cas d'aléa : provisionnement de l'infrastructure, construction et analyse des images, déploiement de l'application, configuration du stockage de sauvegarde.

Ce rôle est transversal par nature : la chaîne que j'ai construite orchestre le travail de tous les autres membres, en assemblant au bon moment l'infrastructure de Zaafir, le chart de déploiement d'Anis, la supervision et les scans de sécurité d'Adame. J'ai donc dû comprendre chaque brique pour l'intégrer correctement, ce qui m'a offert une vue d'ensemble précieuse sur l'articulation de la solution.

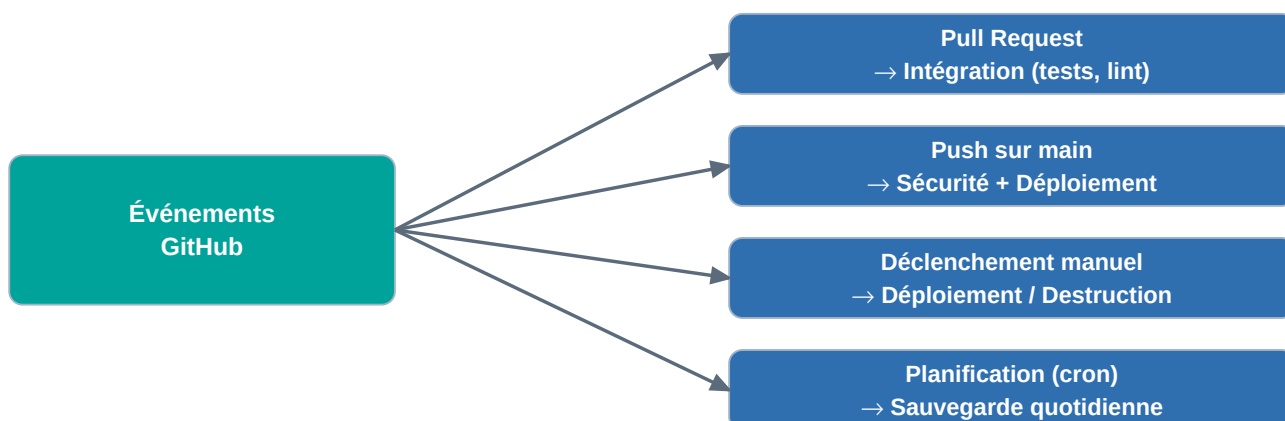
L'automatisation n'est pas une commodité : c'est elle qui rend le projet reproductible, auditable et économe. Un déploiement manuel serait long, source d'erreurs et non traçable ; une chaîne automatisée garantit au contraire que chaque mise en production suit exactement les mêmes étapes, vérifiées et enregistrées, ce qui est une exigence forte du cahier des charges.

2. Ma contribution détaillée

2.1 Architecture de la chaîne GitHub Actions

J'ai conçu plusieurs workflows complémentaires. Le workflow d'intégration valide chaque modification par l'analyse statique du code et l'exécution des tests, côté backend comme côté frontend. Le workflow de sécurité exécute les analyses de vulnérabilités. Le workflow de déploiement orchestre l'ensemble de la mise en production. Enfin, un workflow de sauvegarde, planifié quotidiennement, déclenche la réplication des données vers le site on-premise.

Cette séparation en workflows spécialisés répond à un souci de clarté et d'efficacité : les vérifications rapides — tests, analyses — s'exécutent à chaque proposition de modification, tandis que les opérations lourdes et coûteuses — provisionnement, déploiement — ne sont déclenchées que de manière maîtrisée, sur la branche principale ou manuellement. Ce découpage limite aussi la consommation de minutes d'exécution, ce qui participe indirectement à la sobriété du projet. Le schéma ci-dessous montre comment chaque type d'événement déclenche le workflow approprié :



Chaque événement (proposition de modification, envoi sur la branche principale, action manuelle ou planification) déclenche automatiquement le workflow adapté, sans intervention humaine.

2.2 Automatisation du provisionnement et de l'état

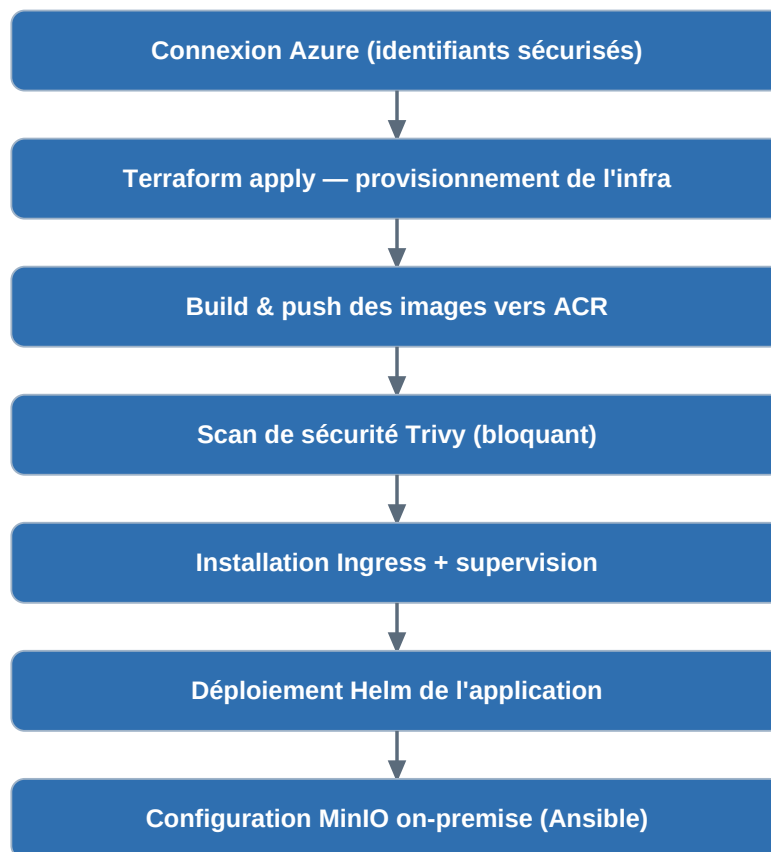
J'ai automatisé la préparation de l'état distant de Terraform : le workflow crée si nécessaire le compte de stockage dédié, puis initialise Terraform sur cet état avant d'appliquer l'infrastructure. Cette étape, souvent laissée manuelle dans les projets, a été entièrement scriptée pour que rien ne doive être préparé à l'avance, ce qui rend le tout premier déploiement aussi simple que les suivants.

L'authentification à Azure au sein de la chaîne s'appuie sur des identifiants stockés dans les secrets chiffrés de GitHub, jamais exposés dans les journaux. J'ai veillé à masquer explicitement les valeurs sensibles — chaînes de connexion, mots de passe — dans les traces d'exécution, afin qu'elles n'apparaissent jamais en clair, même lors d'une session de débogage. Cette discipline est essentielle car les journaux de CI sont souvent consultés et partagés.

2.3 Orchestration et ordonnancement des étapes

Le déploiement complet enchaîne, dans un ordre précis, la connexion à Azure, l'application Terraform, la récupération des informations produites — adresses, identifiants — la construction et la publication des images, leur analyse de sécurité, l'installation des briques du cluster, puis le déploiement de l'application et la configuration du site on-premise.

J'ai dû gérer finement les dépendances entre ces étapes. L'exemple le plus parlant concerne la supervision : les définitions de ressources personnalisées qu'elle introduit doivent être installées **avant** l'application qui les référence, faute de quoi le déploiement échoue. Comprendre et ordonner correctement ces dépendances a représenté une part importante de mon travail. La séquence complète du pipeline est la suivante :



2.4 Optimisation et fiabilisation

J'ai optimisé les temps d'exécution en supprimant des attentes bloquantes inutiles : plutôt que d'attendre que chaque composant de supervision soit totalement prêt, la chaîne applique les manifests et laisse les composants converger en arrière-plan, ce qui a réduit la durée du déploiement de façon sensible tout en garantissant que les éléments réellement indispensables sont bien en place avant de poursuivre.

La partie la plus exigeante a consisté à rendre le pipeline robuste face aux états intermédiaires. Un déploiement interrompu pouvait laisser un verrou sur l'état de l'infrastructure, ou une livraison Helm figée dans un état instable empêchant toute reprise. J'ai introduit des mécanismes qui détectent ces situations et les corrigent automatiquement : libération du verrou résiduel, nettoyage d'une livraison bloquée avant réinstallation. Grâce à ces garde-fous, la chaîne est devenue **auto-réparante** et se remet d'elle-même d'un incident, sans qu'un opérateur ait à intervenir.

2.5 Journal des incidents et correctifs durables

Plutôt que de contourner les échecs au cas par cas, j'ai tenu un véritable journal des incidents rencontrés lors des premières mises en production, en traitant chacun à sa racine par un correctif intégré définitivement au pipeline. Le tableau ci-dessous récapitule les principaux incidents, leur cause profonde et la correction durable apportée :

Incident rencontré	Cause racine	Correctif durable intégré
Action de scan introuvable	Référence de version inexistante	Épinglage sur une référence valide

Incident rencontré	Cause racine	Correctif durable intégré
Namespace déjà existant	Créé par l'outil et par le chart	Création confiée à un seul mécanisme
Verrou d'état résiduel	Déploiement interrompu	Libération automatique avant chaque apply
Livraison Helm bloquée	Release dans un état instable	Détection puis réinstallation propre
Supervision en échec	Définitions de ressources absentes	Réordonnancement : CRD avant l'application
Sauvegarde en échec	Caractère spécial dans le mot de passe	Mot de passe strictement alphanumérique

Chaque correctif a été validé par une exécution complète de la chaîne avant d'être considéré comme acquis. Ce journal illustre concrètement ma démarche : un pipeline fiable ne naît pas d'un seul jet, il se durcit incident après incident, et chaque problème résolu à sa racine bénéficie durablement à l'ensemble de l'équipe.

2.6 Concurrency, déclencheurs et sobriété d'exécution

J'ai également soigné la gestion de la concurrence : une contrainte empêche deux déploiements de s'exécuter simultanément sur le même environnement, ce qui éviterait des états incohérents si deux envois de code se succédaient rapidement. Les déclencheurs ont par ailleurs été calibrés avec soin — envoi sur la branche principale, exécution manuelle avec choix de l'action, ou planification pour les sauvegardes — afin qu'aucun déploiement coûteux ne parte par inadvertance. Cette maîtrise des déclencheurs participe directement à la démarche FinOps de l'équipe, en évitant de consommer inutilement des ressources cloud ou des minutes d'exécution.

3. Choix technologiques : pourquoi ces technologies plutôt que d'autres

Deux décisions ont structuré mon périmètre : le choix de la plateforme d'intégration et de déploiement continu, et le modèle de déploiement (piloté par la chaîne ou par un opérateur GitOps). Je les justifie ci-dessous par comparaison directe avec leurs alternatives.

3.1 Plateforme CI/CD : GitHub Actions plutôt que GitLab CI ou Jenkins

Critère	GitHub Actions (retenu)	GitLab CI	Jenkins
Intégration au dépôt	Native (dépôt sur GitHub)	Native si dépôt GitLab	Externe, à connecter
Hébergement	Runners gérés, sans serveur	Runners gérés ou auto-hébergés	Serveur à installer et maintenir
Catalogue réutilisable	Très large (Marketplace)	Templates	Plugins (maintenance lourde)
Gestion des secrets	Intégrée et chiffrée	Intégrée	Via plugins / credentials
Coût pour le projet	Inclus, minutes gratuites	Inclus si GitLab	Coût d'un serveur permanent

Le cahier des charges évoquait GitLab CI ; le dépôt étant hébergé sur GitHub, j'ai retenu **GitHub Actions**, dont le principe est rigoureusement identique — build, test, analyse, déploiement orchestré. Sa proximité avec le dépôt, son vaste catalogue d'actions réutilisables et sa gestion intégrée des secrets en font l'outil le plus naturel et le plus économe ici. Jenkins, très puissant, aurait imposé d'héberger et de maintenir en permanence un serveur dédié — un coût récurrent et une charge d'exploitation incompatibles avec un budget de 85 dollars. Le choix a donc été pragmatique et sans perte fonctionnelle par rapport à l'outil cité en exemple.

3.2 Modèle de déploiement : approche « push » plutôt que GitOps

Critère	Push par la chaîne (retenu)	GitOps (ArgoCD / Flux)
Composant sur le cluster	Aucun (la CI pousse)	Opérateur permanent à héberger
Simplicité pour un MVP	Élevée, immédiate	Plus complexe à mettre en place
Coût / ressources	Nul en plus	Consomme CPU/RAM en continu
Traçabilité de l'état	Journaux de la chaîne	État réconcilié en continu (supérieur)
Retour arrière	Re-déploiement d'une version	Réconciliation automatique

J'ai privilégié un déploiement piloté par la chaîne, selon une approche « **push** », plutôt qu'un opérateur GitOps, afin de rester simple et économe. Un opérateur tel qu'ArgoCD ou Flux aurait introduit un composant permanent à héberger sur le cluster, consommant des ressources en continu — difficilement justifiable au regard du budget. Le GitOps offre une traçabilité et une réconciliation d'état supérieures, que j'ai clairement identifiées comme la suite logique une fois le projet stabilisé et le budget desserré ; pour un MVP, l'approche push apportait l'essentiel du bénéfice sans le surcoût.

Enfin, le choix de séparer les workflows plutôt que de tout concentrer dans un pipeline unique répond à un principe de responsabilité claire : chaque workflow a un objectif précis, se déclenche sur les bons événements et peut être compris et maintenu indépendamment des autres, ce qui facilite le travail à plusieurs et la reprise du projet par un tiers.

4. Difficultés rencontrées et solutions apportées

J'ai été confronté à une série d'échecs typiques d'une première mise en production : une action dont la version référencée n'existait pas, un conflit lié à une ressource déjà existante, un verrou d'état résiduel après une interruption, une livraison Helm bloquée, un mauvais ordonnancement des dépendances. Plutôt que de les contourner au cas par cas, j'ai choisi de traiter chaque incident à sa racine, en ajoutant dans le pipeline le correctif durable correspondant.

Par exemple, le conflit de création du namespace applicatif — créé à la fois par l'outil et par le chart — a été résolu en confiant cette création à un seul mécanisme. De même, le déploiement de la supervision qui échouait faute de définitions de ressources préalables a été corrigé en réordonnant les étapes. Chacune de ces corrections a été validée par une exécution complète de la chaîne.

Cette approche m'a beaucoup appris : un pipeline robuste ne se conçoit pas d'un seul jet, il se durcit par itérations successives, au contact des situations réelles. Le résultat est une chaîne qui,

aujourd'hui, se déroule de bout en bout de façon fiable, reproductible et rapide, et qui se rétablit seule en cas de problème.

5. Perspectives d'évolution

La première évolution que je viserais est l'adoption du GitOps, avec un outil tel qu'ArgoCD : l'état souhaité du cluster serait décrit dans le dépôt, et un opérateur se chargerait en continu de faire converger le cluster vers cet état, offrant une traçabilité complète et des retours arrière instantanés.

J'introduirais ensuite une authentification fédérée OIDC pour supprimer définitivement les secrets de longue durée, remplacés par des jetons éphémères, ce qui lèverait la contrainte actuelle d'un compte sans double authentification et renforcerait nettement la sécurité de la chaîne.

Je mettrais également en place des environnements distincts — développement, pré-production et production — avec une promotion contrôlée des versions de l'un à l'autre, afin que les essais ne se déroulent plus directement sur l'unique cluster de production.

Enfin, j'ajouterais des tests de bout en bout exécutés automatiquement après chaque déploiement : vérifier, avant d'ouvrir le service, que les parcours utilisateurs essentiels fonctionnent réellement constituerait un filet de sécurité supplémentaire et compléterait naturellement les tests unitaires déjà en place.

6. Analyse critique des limites

Regrouper le provisionnement et le déploiement dans un même workflow est pratique pour la démonstration, mais allonge la durée d'exécution ; un découpage en jobs parallélisés, voire en pipelines distincts pour l'infrastructure et l'application, serait plus efficace à l'échelle et permettrait de ne rejouer que ce qui a changé.

L'authentification par identifiant et mot de passe, retenue pour sa simplicité de mise en route, impose un compte sans double authentification, ce qui n'est pas acceptable dans un contexte de production et constitue la limite de sécurité la plus importante de mon périmètre.

Enfin, l'absence d'environnement de pré-production fait que les essais se déroulent directement sur l'unique cluster, et un déclenchement automatique sur chaque envoi de code pourrait lancer des déploiements coûteux ; un garde-fou explicite — validation manuelle, restriction par branche ou par chemin de fichiers — mériterait d'être renforcé pour éviter toute dépense involontaire.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Configurer les secrets

Le fonctionnement de la chaîne repose sur quelques secrets à renseigner une seule fois dans les paramètres du dépôt : les identifiants de connexion à Azure, l'identifiant de l'abonnement et du tenant, et l'adresse de courriel des alertes budgétaires. Ces valeurs sont chiffrées par GitHub et utilisées par les workflows sans jamais apparaître en clair.

7.2 Lancer un déploiement

Depuis l'onglet Actions, l'utilisateur sélectionne le workflow de déploiement et l'exécute en choisissant l'action de déploiement ; la chaîne se charge de tout et affiche l'adresse d'accès à la dernière étape. Un déploiement peut aussi être déclenché automatiquement par un envoi de code sur la branche principale.

7.3 Détruire l'infrastructure

Le même workflow, exécuté avec l'action de destruction, supprime l'ensemble des ressources et arrête la facturation. L'opération est sûre et reproductible : un déploiement ultérieur reconstruit l'environnement à l'identique en une douzaine de minutes.

7.4 Sauvegarder et restaurer

Le workflow de sauvegarde s'exécute automatiquement chaque jour et peut être déclenché manuellement, en mode sauvegarde ou en mode restauration. Il gère l'export chiffré des données vers le site on-premise et leur réinjection en cas de besoin, sans qu'aucune connaissance technique particulière ne soit requise de l'utilisateur.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de transformer une succession de scripts en une chaîne réellement fiable, capable d'absorber les aléas sans intervention humaine. Chaque échec rencontré était une énigme à résoudre à la racine, ce qui a exigé rigueur, méthode et persévérance.

J'ai aussi dû composer avec le caractère transversal de mon rôle : dépendant du travail de chacun, j'ai appris à coordonner mes évolutions avec celles des autres membres et à communiquer clairement sur l'état de la chaîne, afin de ne pas bloquer l'équipe.

Forces

Je retiens ma rigueur dans l'ordonnancement et l'idempotence des étapes, ainsi que ma capacité à diagnostiquer rapidement l'origine d'un échec de pipeline à partir des journaux d'exécution.

J'ai également su privilégier des corrections durables plutôt que des contournements ponctuels, ce qui a durablement renforcé la fiabilité de l'ensemble et bénéficié à tout le groupe.

Faiblesses

Mes workflows sont encore trop monolithiques et gagneraient à être modularisés en briques réutilisables, ce qui en faciliterait la maintenance et la réutilisation.

Par ailleurs, j'ai concentré mes efforts sur le bon fonctionnement de la chaîne, parfois au détriment de sa propre sécurisation — notamment l'authentification — que je dois encore approfondir.

Compétences développées

J'ai gagné en maîtrise sur GitHub Actions — workflows, secrets, déclencheurs, gestion de la concurrence — ainsi que sur l'automatisation de Terraform et le diagnostic des pipelines.

J'ai surtout compris ce qui distingue une automatisation fragile d'une chaîne réellement industrielle : la capacité à se rétablir seule et à produire un résultat identique à chaque exécution.

Axes d'amélioration personnels

Je souhaite modulariser mes pipelines à l'aide de workflows réutilisables et mettre en place une véritable stratégie multi-environnements avec promotion contrôlée des versions.

Je compte également adopter le GitOps et renforcer mes compétences en sécurité de la chaîne d'approvisionnement logicielle, afin de tendre vers des pratiques de livraison continue de niveau professionnel.